Solution: Min Cost Climbing Stairs

Problem Statement

Given an array of integers `cost`, where `cost[i]` represents the cost of the i^{th} step on a staircase, determine the minimum total cost to reach the top of the staircase.

It is given that once you pay the cost, you can climb the stairs by either taking `one step` or `two steps` at a time. You can start stepping either from the `0` or `1` index.

Examples

1. **Example 1**:
    - **Input**: `[2, 7, 9, 3, 1]`
    - **Expected Output**: 10
    - **Justification**: The minimum cost path starts on the second step (cost=7), then jumps two steps to the fourth step (cost=3), and finally reaches the end, totaling a cost of 10.
2. **Example 2**:
    - **Input**: `[3, 2, 10, 1]`
    - **Expected Output**: 3
    - **Justification**: The minimum cost path is starting on the second step (cost=2) and then jumping two steps to the last step (cost=1), totaling a cost of 3.
3. **Example 3**:
    - **Input**: `[5, 10, 5, 15, 10, 5, 20]`
    - **Expected Output**: 25
    - **Justification**: The optimal path involves taking the first step (cost=5), skipping to the third step (cost=5), then to the fifth step (cost=10), and finally to the sixth step (cost=5), totaling a cost of 25.

Solution

To solve this problem, we'll employ dynamic programming, a strategy that breaks the problem down into smaller subproblems, solves each subproblem just once, and stores their solutions in an array or two variables to avoid redundant calculations. This approach is effective because it exploits the problem's overlapping subproblems and optimal substructure characteristics.

The key idea is to calculate the minimum cost to reach each step from the bottom up, considering the minimum of the two possible previous steps' costs plus the current step's cost. This method ensures that we consider all possible paths to reach a step in an efficient manner, ultimately leading to the minimum total cost to reach the top.

Step-by-step Algorithm

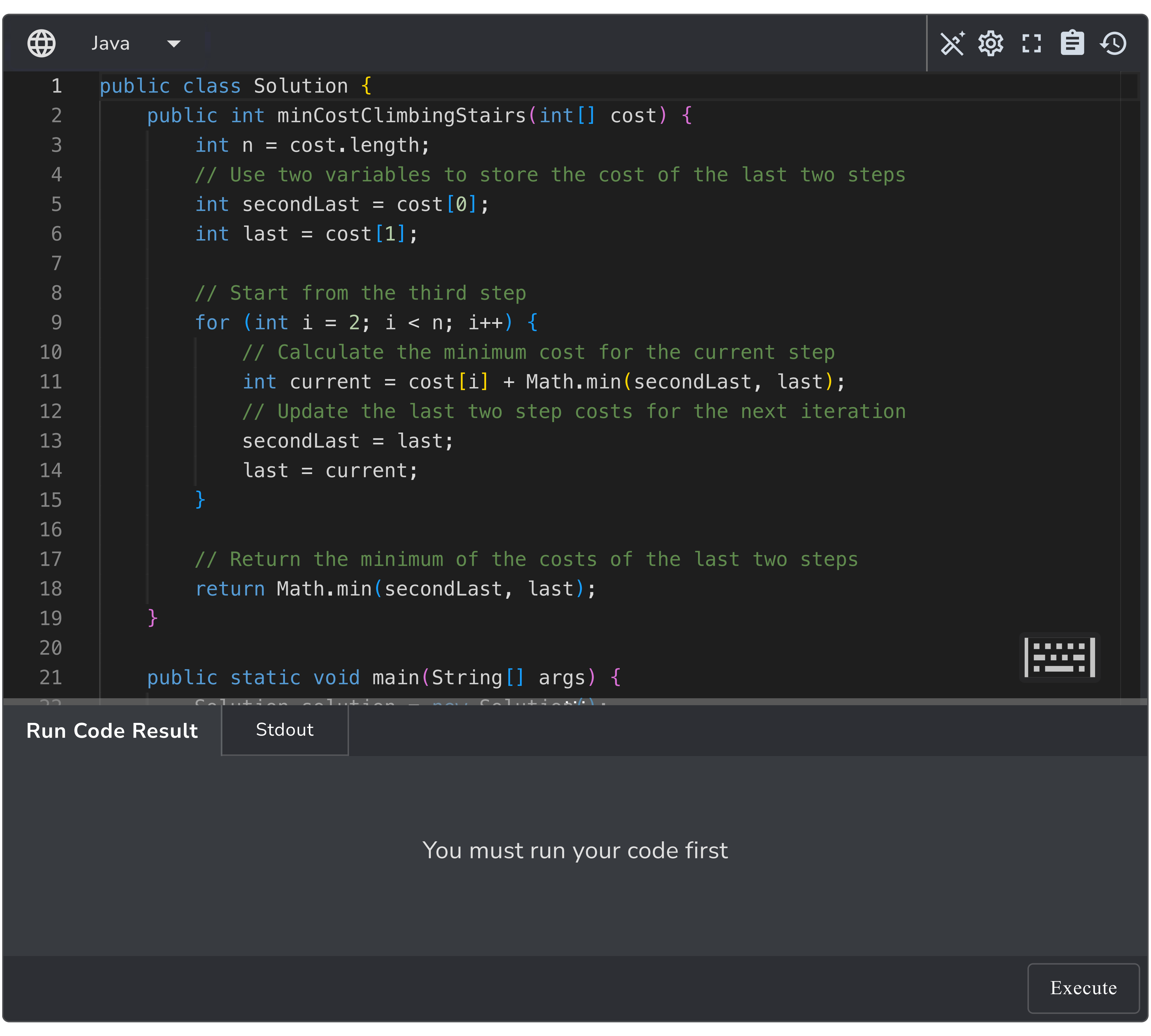
1. **Initialize Two Variables:** Start by initializing two variables, `secondLast` and `last`, to hold the costs of the first two steps, respectively. This is because you can start climbing the stairs from either of the first two steps.
2. **Iterate Through the Array:** Begin iterating from the third step (index 2 in the array) until the last step. For each step `i`, calculate the cost to reach that step as follows:
    - Calculate the cost to reach the current step (`current`) as the sum of the cost of the current step (`cost[i]`) and the minimum cost to reach either of the two preceding steps (`Math.min(secondLast, last)`).
3. **Update Variables:** After calculating the cost for the current step, update `secondLast` to `last`, and `last` to `current`. This step prepares the variables for the next iteration by shifting them to represent the last two steps relative to the next position in the iteration.
4. **Return the Minimum Cost:** After completing the iteration through the array, return the minimum of `secondLast` and `last`. This represents the minimum cost to reach the top of the stairs, as you can finish on either the last step or skip it if it's cheaper to do so.

Algorithm Walkthrough

Let's consider the input: `[5, 10, 5, 15, 10, 5, 20]`

1. **Initialization:**
    - `secondLast = 5` (cost to reach the first step)
    - `last = 10` (cost to reach the second step)
2. **Step 1 (Index 2, Cost = 5):**
    - Calculate `current`: `current = 5 + min(5, 10) = 10`
    - Update `secondLast` and `last`: `secondLast = 10`, `last = 10`
3. **Step 2 (Index 3, Cost = 15):**
    - `current = 15 + min(10, 10) = 25`
    - Update: `secondLast = 10`, `last = 25`
4. **Step 3 (Index 4, Cost = 10):**
    - `current = 10 + min(10, 25) = 20`
    - Update: `secondLast = 25`, `last = 20`
5. **Step 4 (Index 5, Cost = 5):**
    - `current = 5 + min(25, 20) = 25`
    - Update: `secondLast = 20`, `last = 25`
6. **Step 5 (Index 6, Cost = 20):**
    - `current = 20 + min(20, 25) = 40`
    - Update: `secondLast = 25`, `last = 40`
7. **Return the Minimum Cost:** The minimum of `secondLast` and `last` is `min(25, 40) = 25`.

Code



```java
public class Solution {
    public int minCostClimbingStairs(int[] cost) {
        int n = cost.length;
        // Use two variables to store the cost of the last two steps
        int secondLast = cost[0];
        int last = cost[1];

        // Start from the third step
        for (int i = 2; i < n; i++) {
            // Calculate the minimum cost for the current step
            int current = cost[i] + Math.min(secondLast, last);
            // Update the last two step costs for the next iteration
            secondLast = last;
            last = current;
        }

        // Return the minimum of the costs of the last two steps
        return Math.min(secondLast, last);
    }

    public static void main(String[] args) {
```

You must run your code first

Complexity Analysis

- **Time Complexity** $(O(n))$: The algorithm iterates through the input array exactly once. Since the operations within each iteration (calculating the minimum of two integers and updating variables) take constant time, the total time taken scales linearly with the size of the input, denoted as `n`.
- **Space Complexity** $(O(1))$: Space efficiency is significantly improved by using only two variables to keep track of the costs to reach the last two steps. This approach eliminates the need for an additional array to store intermediate results. As a result, the amount of memory used does not depend on the input size, making the space complexity constant.

← Previous
Min Cost Climbing Stairs (Easy)

Next →
Plus One (Easy)

☐ Mark as Completed